k-NN, Naive Bayes and Support Vector Machines — Key Concepts

Lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · a one-page index

Contents

k-nearest neighbours 1
The curse of dimensionality 1
Naive Bayes 2
Support vector machines 3
Choosing 3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

k-nearest neighbours

The algorithm. To classify a new point, find the k training points closest to it and take the majority vote. There is no training step. → § 2.1

Lazy learner. A method whose "fitting" is storing the data. An unusually honest name for an algorithm. → § 2.1

Euclidean distance. The square root of the summed squared differences per feature — which is why every feature must be on a comparable scale before it is used. → § 2.1

k as the bias-variance dial. Small k follows every point including those with flipped labels; large k smooths until it stops following real structure. At $k = 1$ the training accuracy is exactly 1 on any dataset. → § 2.2

Prediction cost. No training time, paid back at every prediction: a naive implementation compares the query against every training row. → § 2.3

The model is the dataset. You cannot ship the model without shipping the training data — a legal question as much as a practical one. → § 2.3

The curse of dimensionality

Distance concentration. As dimensions grow, every distance converges on the same value, and "nearest" stops being a meaningful claim. → § 3.2

Relative spread. The spread of the distances measured against their own size, which is what "nearest" actually needs. Adding dimensions grows the centre faster than the spread, so the ratio vanishes. → § 3.2

What the curse is. Not that high-dimensional problems are unlearnable — that methods built on *distance* lose their footing, because the quantity they depend on stops varying. → § 3.3

Why adding features is not free here. With a linear model an irrelevant feature costs a coefficient near zero. With k-nearest neighbours it costs a dimension in the distance, and dimensions are what the method is made of. → § 3.3

Naive Bayes

Bayes' rule for a class. Turn the probability of a class given the data into a likelihood times a prior, dropping the denominator because it is the same for every class. → § 4.1

The evidence $P(x)$. How likely these readings are whatever the class: the sum of the numerators. Dropped to choose the class, recovered exactly by normalising the scores, so dropping it costs no calibration. → § 4.1

Where the difficulty is. Estimating the probability of an exact combination of readings — a density in as many dimensions as you have features. → § 4.1

The naive assumption. Given the class, the features are independent of one another. → § 4.2

What it buys. One n-dimensional estimation problem becomes n one-dimensional ones, trained in a single pass, needing very little data per feature. → § 4.2

Independence *given the class*. Not the same as independence overall: two features can be correlated across the data and independent inside each class, or the reverse. The distinction is what makes the assumption testable. → § 4.2, § 4.3

Uncorrelated is not independent. A correlation near zero rules out only straight-line dependence; a symmetric one, like a ring, averages to zero and is still there. → § 4.3

Wrong densities, right boundary. A false assumption costs little when the boundary the model implies still has the true shape — on the pumps, an ellipse. → § 4.3

Gaussian Naive Bayes. Each one-dimensional factor is a bell curve, with its own mean and variance for every feature in every class. → § 4.2

Training Naive Bayes. Class priors, then a mean and a variance per feature per class, in one pass: averages, not optimisation, and nothing to converge. → § 4.2

A density is not a probability. A narrow bell peaks above 1, so a log-density can be positive; only the comparison between classes matters. → § 4.2

Where it fails. When the signal is an *interaction* — a rule about the two features together that neither shows alone. No quantity of data repairs it, because the model cannot represent what is being asked. → § 4.4

Uncalibrated probabilities. Its scores can be useful as a ranking while being worthless as probabilities, in either direction. → § 4.5

When to use it anyway. Very high dimensions with little data, and as a baseline that trains in one pass. → § 4.6

Support vector machines

The margin. The width of the slab you can push out from the boundary before it touches the nearest point of either class. → § 5.1

Why maximise it. A boundary close to a training point is one perturbation away from being wrong. The widest margin tolerates the most movement in the data — a statement about generalisation, not about fit. → § 5.1

Support vectors. The points touching the slab. They alone determine the answer; move any other point and nothing changes. → § 5.1

Distance to the boundary. The value of $w^\top x + b$ divided by $\|w\|$ — the division being what removes the freedom to rescale w and b without moving the boundary. → § 5.2

Canonical normalisation. Spending that freedom deliberately, by fixing the scale so the closest points satisfy $|w^\top x + b| = 1$. It is what makes the constraints read ≥ 1 and the margin equal $2/\|w\|$. → § 5.2

Soft margin, and the hinge loss. Violations allowed and charged: nothing for a point outside the slab on its own side, then a straight-line rise. → § 5.2

C, the price of an error. Large C makes violations expensive, so the model accepts a narrower margin to classify more training points; small C buys a wider, calmer boundary. Note the direction: **large C means less regularisation**. → § 5.2

A high support-vector fraction. A free warning that the model is struggling to find room — that no slab separates anything. → § 5.3

Multipliers, and the dual form. One price per training point, charged where its constraint binds. Solving for them shows the boundary to be a weighted sum of the training points, in which the data appears only inside inner products. → § 5.4

Why support vectors are the model. A point strictly outside the slab carries a multiplier of zero, so it drops out of both the fit and the prediction. → § 5.4

The kernel trick. A kernel returns two points' inner product after a lift, computed without lifting; so the lift itself is never needed. → § 5.4

Radial basis function (RBF) kernel. A similarity, 1 for identical points and falling with distance; an infinite-dimensional space for one exponential. → § 5.4

Training and inference. One weight per point, found two at a time with no learning rate; a prediction is the support vectors' similarity-weighted vote. → § 5.5

γ. How far a single training point's influence reaches. Large γ lets the boundary dissolve into islands around individual points, including those with flipped labels. → § 5.6

Choosing

Family before tuning. On the same data the gap between the best and worst family here is larger than any difference this course has shown between a model and a tuned version of itself. → § 6

How to choose. Few features and an odd-shaped boundary, many features and little data, a need for calibrated probabilities, a need for a small fast model — each points at a different one of the three. → § 6